

BALLOO PROJECT BUILD SPECIFICATION (PHASE 1-2)

Ticket ID: BALLOO-BUILD-20260614-001

Версия: 1.0.0

Дата: 2026-06-14

Статус: Active

Дедлайн Phase 1: 2026-06-22

Ответственный: Оберюхтин Иван Анатольевич (o8eryuhtin@yandex.ru)

EXECUTIVE SUMMARY

Цель: Создать 100% работоспособную систему Balloo для Phase 1-2 на одном сервере Ubuntu 22.04.

Ограничения:

- ☒ 1 сервер (Ubuntu 22.04 LTS)
- ☒ Docker + Docker Compose
- ☒ PostgreSQL 15+ + Redis (на том же сервере)
- ☒ Яндекс.Диск для хранения файлов
- ☒ Android SMS-узел для OTP
- ☒ Без платных функций
- ☒ 120,000 пользователей (плановая нагрузка)

Включено в Phase 1-2:

- 7 узлов (working, messenger, admin, kodegen, workdocs, nodes-switcher, api)
 - 3 auth провайдера (yandex-id, email-password, phone-3char-code)
 - 4 роли (creator-superadmin, delegated-node-admin, company-staff, sandbox-operator)
 - ~30 UI компонентов
 - ~50 TypeScript типов
 - 35% minimum test coverage
-

РАЗДЕЛ 1: СЕРВЕРНАЯ ИНФРАСТРУКТУРА

1.1 Серверные параметры

Параметр	Значение
OS	Ubuntu 22.04 LTS
Количество серверов	1
Домен	balloo.su
SSL	Let's Encrypt (Certbot)

Параметр	Значение
IP	Динамический (получать при запуске)

1.2 Базы данных и сервисы

Сервис	Версия	Хост	Порт	Примечания
PostgreSQL	15+	localhost	5432	Основная БД
Redis	7+	localhost	6379	Кэш + сессии
Message Queue	—	—	—	Не используется в Phase 1-2

1.3 Контейнеризация

Технология	Используется	Примечания
Docker	☒ Да	Основной способ деплоя
Docker Compose	☒ Да	Оркестрация сервисов
Kubernetes	☒ Нет	Планируется позже
Registry	Docker Hub + local	Публичные образы + кастомные

1.4 Docker Compose Структура

```
# docker-compose.yml
services:
  postgres:
    image: postgres:15
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    environment:
      POSTGRES_DB: balloo
      POSTGRES_USER: balloo
      POSTGRES_PASSWORD: ${DB_PASSWORD}

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

  balloo:
    build: ./balloo
    ports:
      - "3000:3000"
```

```
depends_on:
  - postgres
  - redis
environment:
  DATABASE_URL: postgresql://balloo:${DB_PASSWORD}@postgres:5432/balloo
  REDIS_URL: redis://redis:6379
  NODE_ID: balloo.su
  NODE_TYPE: client-app

messenger:
  build: ./messenger
  ports:
    - "3001:3000"
  depends_on:
    - postgres
    - redis
  environment:
    DATABASE_URL: postgresql://balloo:${DB_PASSWORD}@postgres:5432/balloo
    REDIS_URL: redis://redis:6379
    NODE_ID: messenger.balloo.su
    NODE_TYPE: client-app

admin:
  build: ./admin
  ports:
    - "3002:3000"
  depends_on:
    - postgres
    - redis
  environment:
    NODE_ID: admin.balloo.su
    NODE_TYPE: client-app

api:
  build: ./api
  ports:
    - "3003:3000"
  depends_on:
    - postgres
    - redis
  environment:
    NODE_ID: api.working.balloo.su
    NODE_TYPE: service

workdocs:
  build: ./workdocs
  ports:
    - "3004:3000"
  environment:
    NODE_ID: workdocs.working.balloo.su
    NODE_TYPE: client-app

kodegen:
```

```

build: ./kodegen
ports:
  - "3005:3000"
environment:
  NODE_ID: kodegen.working.balloo.su
  NODE_TYPE: technical

nodes-switcher:
build: ./nodes-switcher
ports:
  - "3006:3000"
environment:
  NODE_ID: nodes-switcher.working.balloo.su
  NODE_TYPE: technical

working:
build: ./working
ports:
  - "3007:3000"
environment:
  NODE_ID: working.balloo.su
  NODE_TYPE: client-app

volumes:
  postgres_data:
  redis_data:

```

РАЗДЕЛ 2: УЗЛЫ (NODES) ДЛЯ PHASE 1-2

2.1 Приоритет узлов

Priority	Node	Canonical Hostname	Group	Status
1	balloo.su	balloo.su	E (Production)	☒ Include (Main)
2	working	working.balloo.su	D (Sandbox)	☒ Include
3	messenger	messenger.balloo.su	E (Production)	☒ Include
4	admin	admin.balloo.su	B (Company)	☒ Include
5	kodegen	kodegen.working.balloo.su	A (Privileged)	☒ Include
6	workdocs	workdocs.working.balloo.su	B (Company)	☒ Include
7	nodes-switcher	nodes-switcher.working.balloo.su	A (Privileged)	☒ Include
8	api	api.working.balloo.su	D (Sandbox)	☒ Include

Отложено до Phase 3:

- pilot-future.working.balloo.su

- alpha.balloo.su
- apps.alpha.balloo.su
- 2commands.alpha.balloo.su
- files.working.balloo.su
- docs.working.balloo.su
- future.working.balloo.su
- admin.working.balloo.su
- workers.working.balloo.su
- about.working.balloo.su
- apps.working.balloo.su

2.1.1 Balloo.su — Главный узел

balloo.su — это основной производственный узел (Group E), который включает:

Компонент	Описание	Статус
Header	Каноничный хедер из messenger (адаптируется)	☒ Существует
Footer	Каноничный футер из messenger (адаптируется)	☒ Существует
Logo	Логотип Balloo с маскотом	☒ Существует
Company Info	NBS-wt, Екатеринбург, слоган	☒ Существует
Landing Page	Главная страница продукта	☒ Требуется
Navigation	Переключение между узлами	☒ Через nodes-switcher

Адаптация Header/Footer для других узлов:

```
// Header используется во всех узлах с контекстно-зависимым меню
// messenger: полный функционал (чаты, настройки, профиль)
// admin: метрики, управление узлами
// working: sandbox функции
// balloo.su: лендинг, навигация по продукту
```

2.2 Messenger Specifications

Параметр	Значение
Real-time	WebSocket (fallback: polling 5s)
Хранение	PostgreSQL + Яндекс.Диск
Типы сообщений	text, file, image, voice, video
Макс. размер файла	25 МБ (изображения/документы)
Аудио/Видео	До 30 секунд

Параметр	Значение
Шифрование	AES-256 (файлы), TLS 1.3 (трафик)

2.3 Яндекс.Диск Интеграция

```
interface YandexDiskConfig {
    oauthClientId: string;
    oauthSecret: string;
    rootFolder: '/balloo-storage';
    encryption: 'AES-256';
    maxFileSize: 25 * 1024 * 1024; // 25 MB
}

interface FileStorage {
    // Файлы хранятся в папке отправителя
    // Структура: /balloo-storage/{senderId}/{messageId}/{filename}
    senderId: string;
    messageId: string;
    filename: string;
    encryptedPath: string;
    decryptionKey: string; // Хранится отдельно
}
```

РАЗДЕЛ 3: AUTH PROVIDERS (PHASE 1)

3.1 Активированные провайдеры

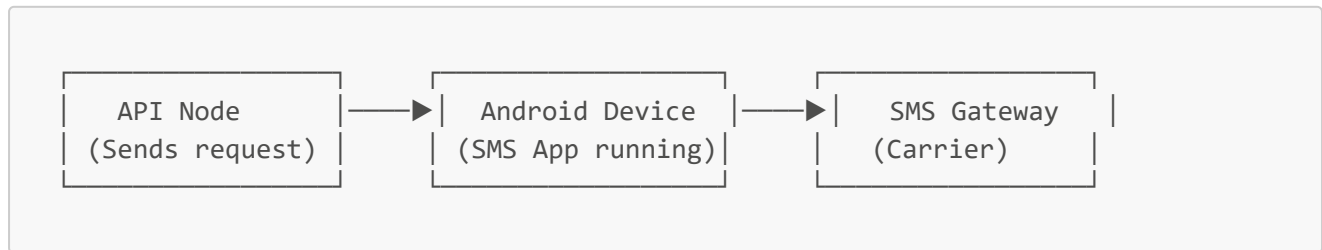
Provider	Type	Status	Identity Anchor
yandex-id	External OIDC	☒ Active	Yandex user ID
email-password	Local credentials	☒ Active	Email
phone-3char-code	Phone OTP	☒ Active	Phone (3-digit, 5min)

3.2 Yandex OAuth Конфигурация

```
interface YandexOAuthConfig {
    clientId: string; // Из Yandex Cloud Console
    clientSecret: string;
    redirectUri: 'https://balloo.su/auth/yandex/callback';
    scopes: ['login:email', 'login:info'];
    authorizationEndpoint: 'https://oauth.yandex.ru/authorize';
    tokenEndpoint: 'https://oauth.yandex.ru/token';
    userInfoEndpoint: 'https://login.yandex.ru/info';
}
```

3.3 Android SMS-узел

Архитектура:



API Endpoint:

```
POST /api/sms/send
{
  "phone": "+79991234567",
  "code": "123",
  "requestId": "uuid-v4"
}

Response:
{
  "status": "sent" | "failed",
  "messageId": "string",
  "timestamp": "ISO-8601"
}
```

Android App Requirements:

- Android 8.0+
- Разрешение: SEND_SMS
- Background service для обработки запросов
- WebSocket connection к API node

3.4 Email Server (Yandex)

```
interface EmailConfig {
  smtpHost: 'smtp.yandex.ru';
  smtpPort: 465; // SSL
  smtpSecure: true;
  auth: {
    user: string; // Yandex email
    pass: string; // App password
  };
  from: 'noreply@balloo.su';
}
```

3.5 Пароль Политика

```
interface PasswordPolicy {
  minLength: 6;
  maxLength: 9;
  allowedChars: {
    lowercase: true;    // a-z
    uppercase: true;    // A-Z
    cyrillic: true;     // а-я, А-Я
    digits: true;       // 0-9
    special: ['_', '=', '+'];
  };
  rules: {
    noRepeatingChars: true; // Запрет "aaa", "111"
    noSimplePasswords: true; // Проверка на "123456", "password"
  };
}

// Валидация
function validatePassword(password: string): boolean {
  // 1. Длина 6-9
  if (password.length < 6 || password.length > 9) return false;

  // 2. Разрешённые символы
  const allowedPattern = /^[a-zA-Za-яA-Я0-9_+=]+$/;
  if (!allowedPattern.test(password)) return false;

  // 3. Нет повторяющихся символов подряд
  if (/(\.)\1\1/.test(password)) return false;

  // 4. Не простой пароль
  const simplePasswords = ['123456', 'password', 'qwerty', '111111'];
  if (simplePasswords.includes(password.toLowerCase())) return false;

  return true;
}
```

РАЗДЕЛ 4: ACCESS ROLES (PHASE 1)

4.1 Активированные роли

Role	Authority Level	Status	Description
creator-superadmin	L10	☒ Always	Оберюхтин Иван Анатольевич
delegated-node-admin	L8	☒	Per-node delegated admin
company-staff	L6	☒	NBS-wt employees
sandbox-operator	L3	☒	Working/sandbox users

Отложено до Phase 3:

- alpha-staff (L5)
- alpha-volunteer (L4)
- public-user (L1)

4.2 Пользователи Phase 1

Параметр	Значение
Плановая нагрузка	120,000 пользователей
Первые пользователи	1,000 сотрудников + 5,000 тестировщиков
Bulk import	CSV + API (HR-системы)

4.3 Доступ к статистике

Role	Доступ к метрикам
creator-superadmin	Полный доступ ко всем метрикам
delegated-node-admin	Полный доступ к метрикам своего узла
company-staff	Просмотр общей загрузки сервера
sandbox-operator	Статистика тестовой среды

4.4 Метрики системы

```
interface SystemMetrics {  
  cpuUsage: number;      // % (0-100)  
  memoryUsage: number;    // MB  
  activeConnections: number; // WebSocket + HTTP  
  smsQueue: number;       // Ожидающие SMS  
  dbLoad: number;         // queries/s  
  diskUsage: number;      // GB (Яндекс.Диск)  
  uptime: number;         // seconds  
  errorRate: number;      // errors/minute  
}
```

РАЗДЕЛ 5: CORE PACKAGES

5.1 Core Packages Usage

Package	Phase 1	Priority	Description
---------	------------	----------	-------------

Package	Phase 1	Priority	Description
core-types	☑	High	~50 типов (common, node, messenger, SMSRequest, SystemMetrics)
core-config	☑	High	Управление настройками узлов
core-i18n	☑	Medium	ru, en
core-theme	☑	Medium	light, dark, russia
core-brand	☑	High	Logo, brand colors, company info (NBS-wt, Екатеринбург)
core-ui	☑	High	~30 компонентов
core-docs-schema	☑	Medium	Структурирование документации

5.2 Core Types (~50 типов)

Категории:

- Common (15): ID, Email, Phone, Timestamp, UUID, etc.
- Node (10): NodeConfig, NodeStatus, NodeHealth, etc.
- Messenger (10): Message, Chat, Attachment, etc.
- Auth (8): User, Session, Role, Permission, etc.
- SMS (4): SMSRequest, SMSResponse, SMSStatus, etc.
- Metrics (3): SystemMetrics, NodeMetrics, UserMetrics

Примеры:

```
// packages/core-types/src/index.ts

// Common
export type ID = string;
export type UUID = string;
export type Email = string;
export type Phone = string;
export type Timestamp = number;

// Node
export interface NodeConfig {
  nodeId: ID;
  nodeType: 'client-app' | 'technical' | 'service';
  hostname: string;
  environment: 'production' | 'alpha' | 'working';
  status: 'online' | 'offline' | 'degraded';
}

export interface NodeStatus {
  nodeId: ID;
```

```

    uptime: number;
    cpuUsage: number;
    memoryUsage: number;
    lastHeartbeat: Timestamp;
}

// Messenger
export interface Message {
    id: UUID;
    chatId: UUID;
    senderId: UUID;
    type: 'text' | 'file' | 'image' | 'voice' | 'video';
    content: string;
    attachments?: Attachment[];
    timestamp: Timestamp;
    encrypted?: boolean;
}

export interface Attachment {
    id: UUID;
    type: 'file' | 'image' | 'audio' | 'video';
    filename: string;
    size: number; // bytes
    yandexDiskPath: string;
    encryptionKey?: string;
}

// Auth
export interface User {
    id: UUID;
    email?: Email;
    phone?: Phone;
    roles: Role[];
    createdAt: Timestamp;
    lastLogin?: Timestamp;
}

export type Role =
    | 'creator-superadmin'
    | 'delegated-node-admin'
    | 'company-staff'
    | 'sandbox-operator';

// SMS
export interface SMSRequest {
    phone: Phone;
    code: string; // 3-digit
    requestId: UUID;
    expiresAt: Timestamp; // +5 minutes
}

export interface SMSResponse {
    status: 'sent' | 'failed';
}

```

```

    messageId: string;
    timestamp: Timestamp;
  }

  // Metrics
  export interface SystemMetrics {
    cpuUsage: number;      // %
    memoryUsage: number;   // MB
    activeConnections: number;
    smsQueue: number;
    dbLoad: number;        // queries/s
    diskUsage: number;     // GB
    uptime: number;        // seconds
    errorRate: number;     // errors/minute
  }

```

5.3 Core Brand (Phase 1)

Компоненты:

Компонент	Описание	Статус
Logo	Логотип Balloo с маскотом	☒ Существует в messenger
Brand Colors	Russia flag (white, blue, red)	☒ Существует в core-brand
Company Info	NBS-wt, Екатеринбург	☒ Требуется обновить
Slogan	"Системы для Ваших Новых Начинаний."	☒ Существует в Footer
Brand Guidelines	Лого clear space: 8px, min size: 32px	☒ Существует

Company Information:

```

// packages/core-brand/src/brand.ts
export const COMPANY_INFO = {
  name: 'NBS - web-tech',
  shortName: 'NBS-wt',
  city: 'Екатеринбург',
  slogan: 'Системы для Ваших Новых Начинаний.',
  year: new Date().getFullYear(),
};

export const BRAND_COLORS = {
  primary: '#0039A6',    // Russia blue
  secondary: '#D52B1E',  // Russia red
  accent: '#007bff',     // Modern blue
  white: 'ffffff',
  blue: '#0039A6',
  red: '#D52B1E',
};

```

Header/Footer Architecture:

```
messenger/src/components/
├─ Header.tsx          # Каноничный хедер (адаптируется для всех узлов)
├─ Footer.tsx          # Каноничный футер (адаптируется для всех узлов)
├─ layout/
│   ├── Header.css      # Стили хедера
│   └── Footer.css      # Стили футера
└─ ui/
    ├── BurgerMenu.tsx  # Бургер меню с маскотом
    └── ThemeSelector.tsx # Выбор тем

packages/core-brand/src/
├─ Logo.tsx            # Logo компонент
├─ brand.ts            # Brand constants
├─ types.ts            # Brand types
└─ index.ts            # Exports
```

Адаптация для узлов:

Узел	Header Menu	Footer Links	Special Features
balloo.su	Лендинг, продукт, компания	Все ссылки	Главный лендинг
messenger	Чаты, настройки, профиль	Стандартные	WebSocket, real-time
admin	Метрики, узлы, пользователи	Стандартные	Stats Dashboard
working	Sandbox функции	Стандартные	Тестовая среда
kodegen	Codegen инструменты	Стандартные	AI codegen
workdocs	Документы	Стандартные	Documentation
nodes-switcher	Переключение узлов	Стандартные	Node navigation
api	API docs	Стандартные	API endpoints

```
### 5.4 Core UI Components (~30 компонентов)

**Категории:**

| Категория | Компоненты | Count |
|-----|-----|-----|
| **Buttons** | Button, IconButton, ToggleButton | 3 |
| **Forms** | Input, TextArea, Select, Checkbox, Radio, PasswordInput | 6 |
| **Layout** | Container, Grid, Flex, Card, Modal, Drawer | 6 |
| **Data Display** | Table, List, ListItem, Badge, Avatar, Tooltip | 6 |
| **Navigation** | Header, Footer, Sidebar, Breadcrumb, Tabs | 5 |
```

```

| **Special** | StatsDashboard, SMSStatusWidget, RealTimeStats,
NodeStatusBlock, LogViewer | 5 |
| **Total** | | **30** |

**Примеры специальных компонентов:**
```tsx
// StatsDashboard
interface StatsDashboardProps {
 metrics: SystemMetrics;
 refreshInterval?: number; // ms
}

// SMSStatusWidget
interface SMSStatusWidgetProps {
 status: 'pending' | 'sent' | 'failed';
 phone: Phone;
 retryCount?: number;
}

// RealTimeStats
interface RealTimeStatsProps {
 nodeId: ID;
 metrics: NodeMetrics;
 updateStream: WebSocket;
}

```

## 5.5 Core I18N

**Языки:** ru, en

**Структура:**

```

packages/core-i18n/
├── locales/
│ ├── ru.json
│ └── en.json
├── index.ts
└── types.ts

```

**Пример:**

```

{
 "common": {
 "loading": "Загрузка...",
 "error": "Ошибка",
 "success": "Успешно"
 },
 "auth": {

```

```

 "login": "Войти",
 "logout": "Выйти",
 "phoneOtp": "Код из SMS"
 },
 "messenger": {
 "sendMessage": "Отправить",
 "fileTooLarge": "Файл слишком большой (макс. 25 МБ)"
 }
}

```

## 5.6 Core Theme

**Темы:** light, dark

**Design Tokens:**

```

// packages/core-theme/src/tokens.ts
export const tokens = {
 colors: {
 primary: '#0066FF',
 secondary: '#6B7280',
 success: '#10B981',
 warning: '#F59E0B',
 error: '#EF4444',
 },
 border: {
 radius: 0, // Design invariant #1
 width: '2px', // Design invariant #2
 style: 'solid',
 },
 spacing: {
 unit: 8, // 8px grid
 touchTarget: 44, // Design invariant #6
 },
 fonts: {
 family: 'system-ui, -apple-system, sans-serif', // Design invariant #4
 },
};

export const themes = {
 light: {
 background: 'FFFFFF',
 surface: 'F3F4F6',
 text: '111827',
 textSecondary: '6B7280',
 },
 dark: {
 background: '111827',
 surface: '1F2937',
 text: 'F9FAFB',
 },
};

```

```

 textSecondary: '#9CA3AF',
 },
 russia: {
 // Special theme for Russia nodes
 background: '#FFFFFF',
 primary: '#0039A6',
 accent: '#D52B1E',
 },
};

```

## ● РАЗДЕЛ 6: CODEGEN

### 6.1 Приоритеты генерации

Priority	Type	Source	Output
1	TypeScript types	core-types contracts	packages/core-types/src/*.ts
2	API routes	endpoint specs	apps/*/src/routes/*.ts
3	Configuration files	schemas	configs/*.json
4	React components	design system	packages/core-ui/src/*.tsx
5	Documentation pages	contracts	SUMMARY_DOCS/pages/*.md
5	Test files	contracts	packages/*/tests/*.test.ts

### 6.2 Codegen Workflow

```

.codegenrc.yml
trigger: pre-commit
commitGenerated: true
templateLocation: templates/
outputs:
 - types
 - api-routes
 - configs
 - components
 - docs
 - tests

```

#### Pre-commit hook:

```

#!/bin/bash
.git/hooks/pre-commit

echo "Running codegen..."

```



```
npm run codegen

Check if any files were modified
if ! git diff --cached --quiet; then
 echo "Generated files updated. Adding to commit..."
 git add -A
fi
```

## 6.3 Templates Structure

```
templates/
├── types/
│ ├── interface.hbs
│ ├── type.hbs
│ └── enum.hbs
├── api-routes/
│ ├── route.hbs
│ └── handler.hbs
├── components/
│ ├── component.hbs
│ └── component.styles.hbs
└── configs/
 ├── node-config.hbs
 └── docker-compose.hbs
```

# РАЗДЕЛ 7: TESTING

## 7.1 Test Types

Type	Framework	Status	Coverage
<b>Unit tests</b>	Jest	☑	core-packages, utils
<b>Integration tests</b>	Supertest	☑	API endpoints, SMS-node
<b>E2E tests</b>	Playwright	✗ Phase 3	—
<b>API tests</b>	Supertest + Jest	☑	messenger, auth, sms-node

## 7.2 Coverage Target

Metric	Target
<b>Minimum coverage</b>	35%
<b>Critical modules</b>	messenger, auth, sms-node, core-types

## 7.3 Test Structure

```

packages/
├── core-types/
│ └── tests/
│ ├── types.test.ts
│ └── validation.test.ts
├── core-config/
│ └── tests/
│ ├── config.test.ts
│ └── validation.test.ts
└── core-ui/
 └── tests/
 ├── Button.test.tsx
 └── Input.test.tsx

apps/
├── messenger/
│ └── tests/
│ ├── api.test.ts
│ └── websocket.test.ts
└── admin/
 └── tests/
 ├── metrics.test.ts
 └── auth.test.ts

```

## ● РАЗДЕЛ 8: DEPLOYMENT (UBUNTU 22.04)

### 8.1 Pre-deployment Checklist

```

1. Update system
sudo apt update && sudo apt upgrade -y

2. Install Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER

3. Install Docker Compose
sudo curl -L
"https://github.com/docker/compose/releases/latest/download/docker-
compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose

4. Install Node.js 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

5. Install PostgreSQL 15
sudo apt install -y postgresql-15 postgresql-contrib

```

```
6. Install Redis
sudo apt install -y redis-server

7. Install Certbot
sudo apt install -y certbot python3-certbot-nginx
```

## 8.2 PostgreSQL Setup

```
1. Create database and user
sudo -u postgres psql

CREATE DATABASE balloo;
CREATE USER balloo WITH PASSWORD 'secure-password';
GRANT ALL PRIVILEGES ON DATABASE balloo TO balloo;
\q

2. Configure pg_hba.conf
sudo nano /etc/postgresql/15/main/pg_hba.conf
Add: host balloo balloo 127.0.0.1/32 md5

3. Restart PostgreSQL
sudo systemctl restart postgresql
```

## 8.3 Redis Setup

```
1. Configure Redis
sudo nano /etc/redis/redis.conf
Set: bind 127.0.0.1
Set: protected-mode yes

2. Restart Redis
sudo systemctl restart redis
```

## 8.4 SSL Certificate (Let's Encrypt)

```
1. Obtain certificate
sudo certbot --nginx -d balloo.su -d www.balloo.su

2. Auto-renewal
sudo certbot renew --dry-run
```

## 8.5 Docker Compose Deployment

```
1. Clone repository
git clone https://github.com/your-org/balloo.git
cd balloo

2. Create .env file
cp .env.example .env
nano .env

.env contents:
DB_PASSWORD=secure-password
YANDEX_CLIENT_ID=your-client-id
YANDEX_CLIENT_SECRET=your-client-secret
SMTP_PASSWORD=your-app-password
CREATOR_EMAIL=o8eryuhtin@yandex.ru
CREATOR_PHONE=+79292167585

3. Build and start
docker-compose up -d --build

4. Check status
docker-compose ps
docker-compose logs -f
```

## 8.6 Android SMS-узел Setup

```
1. Build Android APK
cd android-sms-node
npm install
npx react-native build-android --release

2. Install on Android device
adb install app/build/outputs/apk/release/app-release.apk

3. Configure app
- Open app
- Enter API endpoint: https://api.working.balloo.su/sms
- Enter auth token
- Grant SMS permissions

4. Start background service
- App runs as background service
- Listens for WebSocket messages
- Sends SMS via Android SMS API
```

## 8.7 Яндекс.Диск Setup

```
1. Create Yandex OAuth app
https://oauth.yandex.ru/client/new

2. Configure permissions
- login:email
- login:info
- disk:app_folder.files

3. Get credentials
- Client ID
- Client Secret

4. Add to .env
YANDEX_CLIENT_ID=your-client-id
YANDEX_CLIENT_SECRET=your-client-secret

5. Create app folder
- First OAuth login creates /balloo-storage folder
```

---

## 🕒 РАЗДЕЛ 9: MONITORING & METRICS

### 9.1 Stats Dashboard (admin node)

**Endpoint:** GET /api/metrics/system

**Response:**

```
{
 "cpuUsage": 45.2,
 "memoryUsage": 2048,
 "activeConnections": 1250,
 "smsQueue": 5,
 "dbLoad": 150,
 "diskUsage": 125.5,
 "uptime": 86400,
 "errorRate": 0.5
}
```

### 9.2 Real-time Stats (WebSocket)

**Endpoint:** WS /ws/metrics

**Message format:**

```
{
 "type": "metrics:update",
```

```

 "payload": {
 "nodeId": "admin",
 "timestamp": 1686744000000,
 "metrics": { ... }
 }
 }
}

```

### 9.3 Alert Thresholds

Metric	Warning	Critical
CPU Usage	> 70%	> 90%
Memory Usage	> 80%	> 95%
Active Connections	> 10,000	> 50,000
SMS Queue	> 100	> 500
Error Rate	> 5/min	> 20/min

## РАЗДЕЛ 10: LEGACY CLEANUP

### 10.1 Directories to Handle

Directory	Action	Phase
docs-contracts/	Archive	Phase 3
docs-migration/	Archive	Phase 3
docs-site/	Delete	Phase 3
workdocs/legacy-*	Delete	Phase 3

### 10.2 Redirect Map

```

{
 "redirects": [
 { "from": "/docs/*", "to": "/SUMMARY_DOCS/*" },
 { "from": "/workdocs/*", "to": "/working.balloo.su/*" }
]
}

```

## 📎 ПРИЛОЖЕНИЯ

A: Полная карта узлов (20 total)

Group A (Privileged, 4):

- ☒ projectgeneralsettings.working.balloo.su (Phase 1)
- ☒ kodegen.working.balloo.su (Phase 1)
- ☒ pilot-future.working.balloo.su (Phase 3)
- ☒ nodes-switcher.working.balloo.su (Phase 1)

Group B (Company, 2):

- ☒ workdocs.working.balloo.su (Phase 1)
- ☒ admin.balloo.su (Phase 1)

Group C (Alpha, 3):

- ☒ alpha.balloo.su (Phase 3)
- ☒ apps.alpha.balloo.su (Phase 3)
- ☒ 2commands.alpha.balloo.su (Phase 3)

Group D (Sandbox, 9):

- ☒ working.balloo.su (Phase 1)
- ☒ api.working.balloo.su (Phase 1)
- ☒ files.working.balloo.su (Phase 3)
- ☒ docs.working.balloo.su (Phase 3)
- ☒ future.working.balloo.su (Phase 3)
- ☒ admin.working.balloo.su (Phase 3)
- ☒ workers.working.balloo.su (Phase 3)
- ☒ about.working.balloo.su (Phase 3)
- ☒ apps.working.balloo.su (Phase 3)

Group E (Production, 2):

- ☒ balloo.su (Phase 3)
- ☒ messenger.balloo.su (Phase 1)

## B: Timeline

Phase	Start	End	Deliverables
Phase 1	2026-06-14	2026-06-22	Core nodes, auth, messenger, SMS, stats
Phase 2	2026-06-23	2026-07-07	E2E encryption, media optimization, SMS scaling
Phase 3	2026-07-08	TBD	Alpha nodes, public access, premium features

## ☒ ACCEPTANCE CRITERIA

Phase 1 Complete When:

- ☐ 7 узлов развёрнуты и работают
- ☐ 3 auth провайдера активны
- ☐ 4 роли работают
- ☐ Messenger отправляет/получает сообщения
- ☐ Файлы сохраняются на Яндекс.Диск

- ☐ SMS-узел отправляет OTP
- ☐ Stats Dashboard показывает метрики
- ☐ 35%+ test coverage
- ☐ Deploy инструкция работает на Ubuntu 22.04

#### Phase 2 Complete When:

- ☐ E2E шифрование для частных чатов
- ☐ Видео/аудио конвертация в WebM
- ☐ Автоматическое масштабирование SMS-узлов
- ☐ 50%+ test coverage

---

### 🔗 Balloo - Переверни общение!

**Создано:** 2026-06-14

**Версия:** 1.0.0

**Статус:** Active

**Автор:** Koda (NLP-Core-Team)

**Дедлайн Phase 1:** 2026-06-22